

CarCommunity Maintenance Center - Project Documentation

This document is intended for software engineers, DevOps engineers, and technical product managers responsible for building and maintaining the multi-tenant CarCommunity platform (ASP.NET Core + Angular).

The document provides a technical overview, architectural details, and quick-start guidelines to facilitate understanding and work on the project. The platform is designed to provide integrated services for car owners and service providers.

Project Overview and Key Features

The Car Community Maintenance Center is a comprehensive platform that allows car owners, maintenance shops, and spare parts sellers to manage vehicles, schedules, community interactions, and business transactions.



Vehicle Management

Registering cars, uploading photos, and viewing detailed maintenance history for each vehicle.



Maintenance Scheduling and Reminders

Scheduling services, with automatic reminders via email and SMS.



E-Marketplace (Marketplace)

Buying and selling spare parts, payment via Stripe, and efficient order tracking.



Augmented Reality (AR) Experiences

Pointing the phone towards the car to view live diagnostics, maintenance tips, and part details.

The platform features Multi-tenant support where each client gets an isolated subdomain (e.g., {tenant}.mycarcommunity.com), and full support for Dark-Mode and Arabic language (RTL).

Key Architectural Components

The platform is built on a modern model focusing on scalability and reliability. All services are centered around an API Gateway to ensure security and proper request routing.



Prerequisites and Tools

To begin working on the project locally, a specific set of tools must be installed to ensure environment compatibility with ASP.NET Core 9 and Angular 18 requirements.



.NET SDK 9.0

Or .NET 8 LTS with
TargetFramework modification.



Node.js 20.x LTS

Required to run and build Angular applications.



Docker Desktop and Docker Compose v2

To run local dependent services (databases, Redis, etc.).



Visual Studio 2022 / VS Code

Recommended development environment.



Terraform 1.9+ and Helm 3.14+

For infrastructure automation and deployment (optional for local work).

Getting Started - Local Setup and Development

Local setup requires an environment file (.env) containing the necessary secret keys to run services like the database, Stripe, and OpenAI, followed by running Docker Compose.

Step 4.3: Build and Run with Docker Compose

The backend (API) runs on port 5000, and the web interface (SPA) on port 4200. SQL Server and Redis databases are automatically provisioned.

Basic running commands:

```
git clone https://github.com/yourorg/car-community-saas.gitcd car-community-saas# Step 4.3: Build and run servicesdocker compose up -d --build
```

Check Backend (Swagger): <http://localhost:5000/swagger>

Check User Interface (SPA): <http://localhost:4200>



Important Note: You must run `docker compose exec api dotnet ef database update` only once to apply database migrations on the first run.



Backend: ASP.NET Core 9

The backend project structure follows a standard pattern, focusing on tenant isolation via `ITenantEntity` and utilizing EF Core global query filters.

Key concepts governing the backend:

Tenant Isolation

Every data entity includes a `TenantId`, and a filter is applied to ensure each tenant accesses only their own data: `HasQueryFilter(e => EF.Property(e, "TenantId") == GetCurrentTenantId());`

Feature Flags Management

The `FeatureFlag` table defines the features available to each tenant based on their `Plan`. Templates are copied when a new tenant is provisioned.

Provisioning Service

Responsible for creating the tenant, Stripe customer, admin user, copying initial feature settings, and sending the welcome email.

Real-time Conversations

`AiChatHub` and `HumanChatHub` (depending on features) are used with `PresenceHub` to track user status and send notifications.

Frontend: Angular 18

The user interface is built using Angular 18 and relies on the `NgRx` pattern for state management. The interface is designed to support advanced features such as Augmented Reality (AR), multilingualism (RTL with Arabic), and dark mode.

`FeatureFlagService` **service:** Queries `/api/tenant/flags` and exposes the `can(key)` function to control the visibility of features based on the tenant's plan.

`ThemeService` **service:** Toggles light/dark mode and stores the preference locally.

`I18nService` **service:** Uses `ngx-translate` to switch languages and set `document.dir` to support Right-to-Left (RTL) writing.

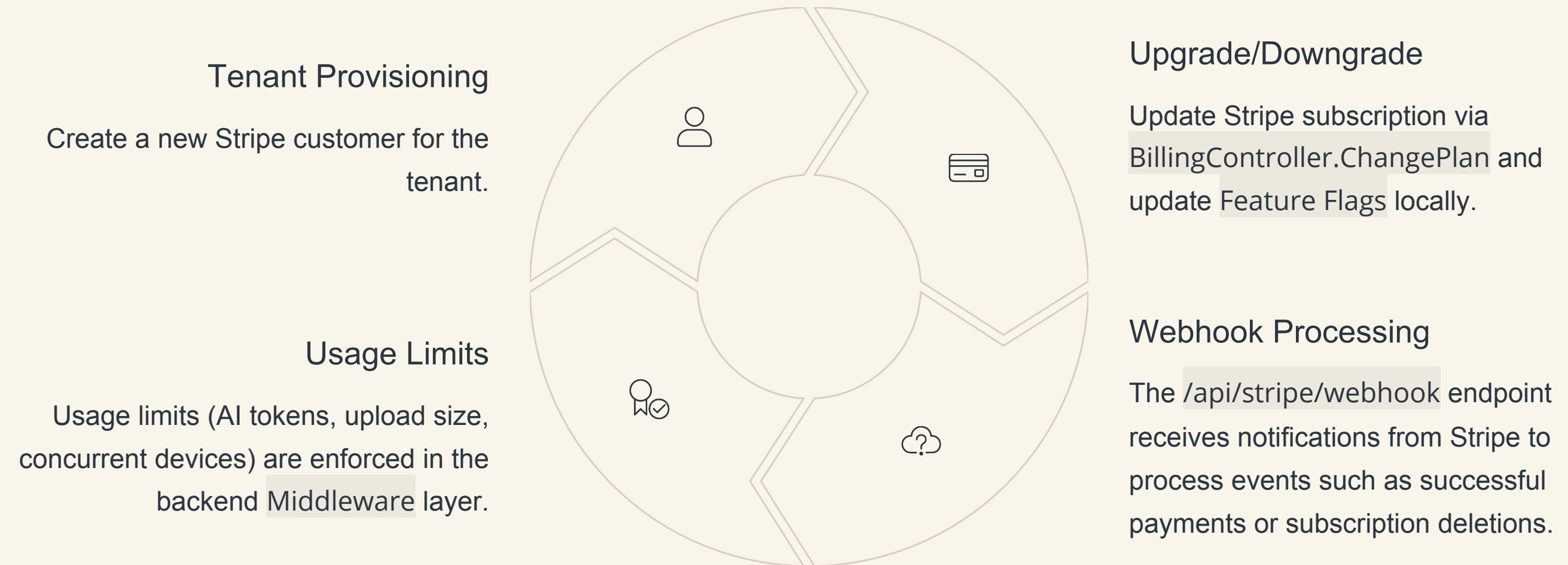
`ArService` **service:** Manages the `WebXR` lifecycle and loads `GLTF` models for Augmented Reality display.

Specific pages such as `/chat/human` and `/ar/car/:id` are protected via Guards that verify feature authorization through the `FeatureFlagService`.



Plans and Invoicing - Stripe Integration

Stripe is used to manage subscriptions and invoicing, ensuring secure and PCI-compliant processing of payment data.



Example of monthly AI token usage limits: `Free 10,000`, `Pro 100,000`, `Enterprise 1,000,000`.

Infrastructure as Code (IaC) and CI/CD

We use Terraform and Helm to manage infrastructure resources and reliable deployment, ensuring identical and repeatable environments.

Terraform and Helm

Terraform is used to provision core cloud resources (Azure AKS, Azure SQL, Key Vault). Helm is used to deploy API and SPA applications, with reference management for secrets.

- Provisioning a resource group (AKS, Azure SQL, ACR, Front Door).
- Embedding secret references from Key Vault into Helm values.

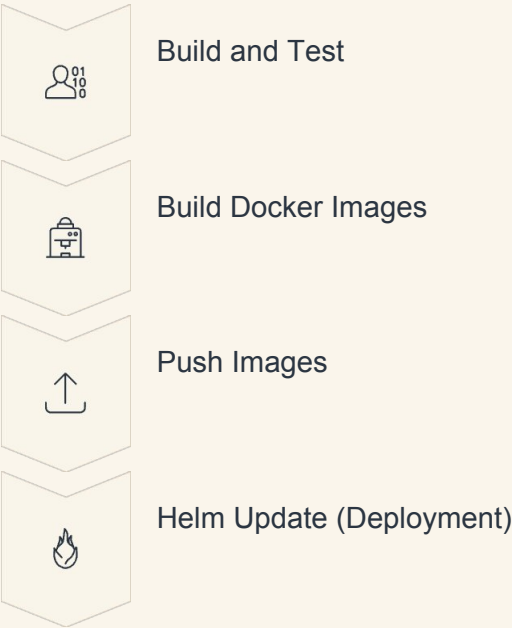
Core commands: `terraform apply` and `helm upgrade --install`



GitHub Actions Workflow

Build, test, and deployment are automated via GitHub Actions workflows, ensuring consistent deployment when merging changes or creating Release Tags.

Workflow Stages:



Monitoring, Logging, and Security

Comprehensive monitoring and robust security measures are essential for a multi-tenant SaaS platform. We use an integrated suite of tools to ensure visibility and compliance.



Monitoring & Metrics

We use Application Insights for tracing and telemetry, along with Prometheus and Grafana for metrics and custom dashboards.



Security & Compliance

Short-lived JWT authentication, role-based authorization (RBAC) and feature flags, per-tenant/IP rate limiting, and PCI/GDPR compliance through no card data storage and automated scanning.

To ensure security, sensitive secrets are stored in Azure Key Vault, TDE is applied to Azure SQL, and TLS everywhere is enforced via Front Door.



Structured Logging

Serilog logs are directed to Seq/Elastic, with logs enriched with `TenantId` and `UserId` for easy traceability.



Alerting

Azure Monitor alerts are configured to detect high error rates, Hangfire job failures, and service health issues via the `/healthz` endpoint.